

Electronic Design Workshop Project

Cloud Based Weather Station

Vansh Singh - 2023UEC2550
Tarun Kumar Sahu - 2023UEC2551
Sumit Kumar - 2023UEC2554

May 2025

Contents

- Acknowledgment
- Introduction
- Materials Required
- Working
- Schematic Diagram
- Arduino IDE Code
- Website Walkthrough
- Website Development
- Conclusion and Learning

Acknowledgement

We are deeply grateful to our mentor, Prof. D.V. Gadre, for his invaluable guidance and encouragement throughout the course of this project.

We would also like to acknowledge the support of the lab staff and the collaborative efforts of our peers, which played a significant role in the successful completion of this work.

This project has been an enriching experience, significantly enhancing our knowledge of embedded systems and the practical aspects of sensor integration.

Introduction

In recent years, the demand for real-time environmental monitoring has grown rapidly due to increasing concerns over climate change, air quality, and weather unpredictability. As part of our Electronic Design Workshop, we have developed a Cloud-Based Weather Station—a compact, low-cost embedded system capable of measuring and displaying critical atmospheric parameters.

This project leverages the capabilities of the ESP8266 microcontroller, paired with various environmental sensors such as the DHT11 and BMP180. The system collects real-time data including temperature, humidity, atmospheric pressure, and altitude, and displays it on a 1.3-inch OLED screen.

Furthermore, with its built-in WiFi capability, the ESP8266 enables future expansion into Internet of Things (IoT) applications—allowing for cloud-based data logging, remote monitoring, and smart environmental control systems.

This report outlines the components used, the system design, schematic, working code, and implementation timeline. It also demonstrates how a simple yet powerful embedded system can contribute meaningfully to smart city infrastructure and educational IoT applications.

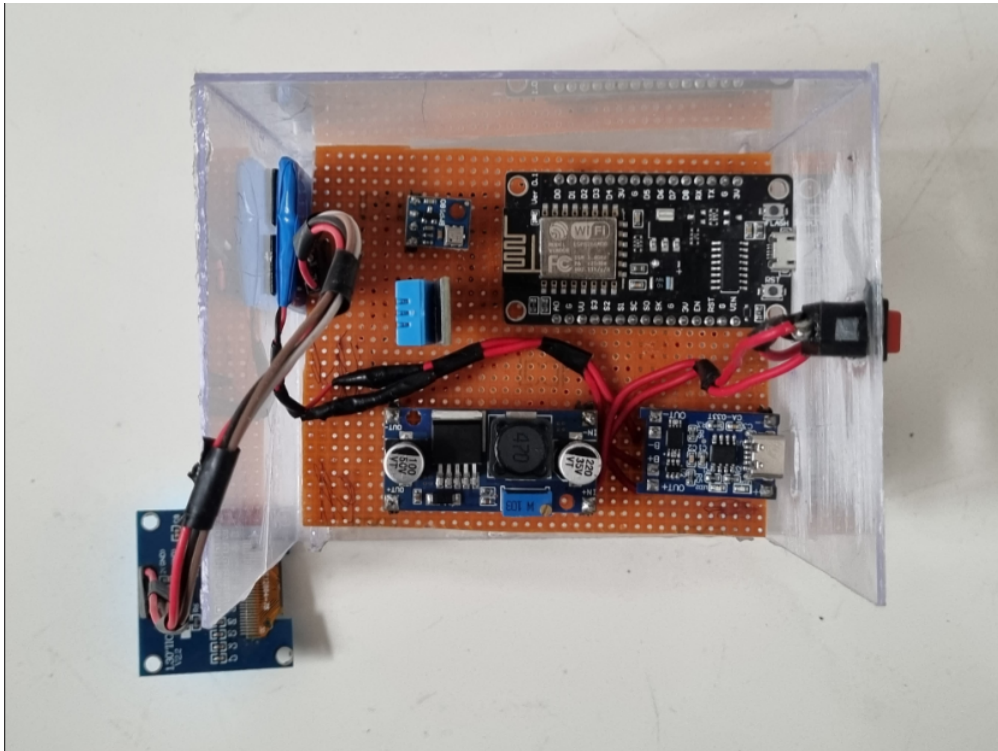


Figure 1: Weather Station

Materials Required

- ESP8266 Microcontroller
- DHT11 Temperature and Humidity Sensor
- BMP180 Barometric Pressure and Altitude Sensor
- OLED Display (1.3 inch)
- TP4056 Lithium-ion Battery Charging Module
- MT3608 Boost Converter
- Jumper Wires
- Zero PCB
- Switch
- 3.7V Li-Ion Battery
- Enclosure Box

DHT11 Sensor

The DHT11 sensor is a widely used low-cost digital sensor for measuring ambient temperature and relative humidity.

It combines a capacitive humidity sensor and a thermistor to sense the surrounding environment, providing digital output signals that can be easily read by the microcontroller.

In our design, the DHT11 is powered by a stable 3.3V supply from the ESP8266, and its data output is connected directly to the microcontroller's GPIO pin for real-time temperature and humidity monitoring.

The sensor's simple one-wire interface and reliable performance make it suitable for embedded weather monitoring systems, providing essential environmental data with minimal complexity.

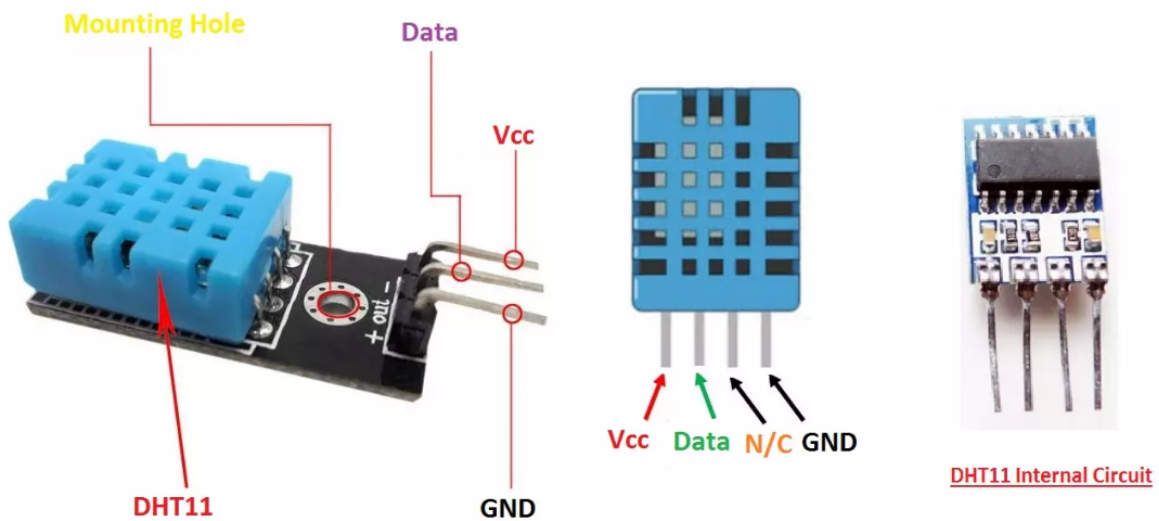


Figure 2: DHT11 Temperature and Humidity Sensor

BMP180 Sensor

The BMP180 sensor is a compact and precise digital barometric pressure sensor capable of measuring atmospheric pressure and calculating altitude based on pressure changes. It communicates with the microcontroller using the I2C protocol, enabling efficient data transfer over two lines.

In our weather station design, the BMP180 is powered by the 3.3V supply and interfaces with the ESP8266 microcontroller to provide real-time pressure and altitude data.

This sensor plays a critical role in environmental monitoring by allowing the system to track changes in atmospheric pressure and elevation accurately.

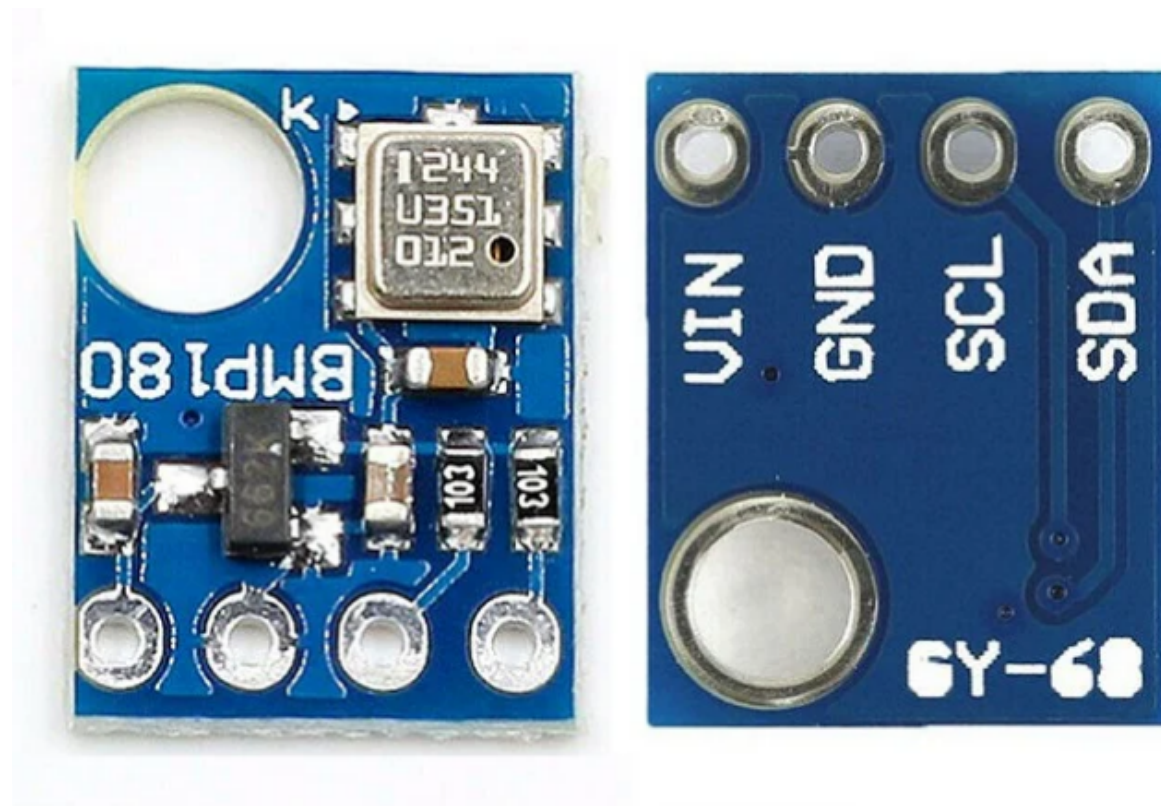


Figure 3: BMP180 Pressure Sensor

ESP8266 Microcontroller

The ESP8266 is a powerful microcontroller with integrated WiFi and Bluetooth, dual-core processing, and multiple GPIO pins.

It acts as the brain of the weather station, collecting sensor data, processing it, and displaying results on the OLED screen.

Its wireless capabilities also enable potential IoT applications such as remote data monitoring and cloud integration.

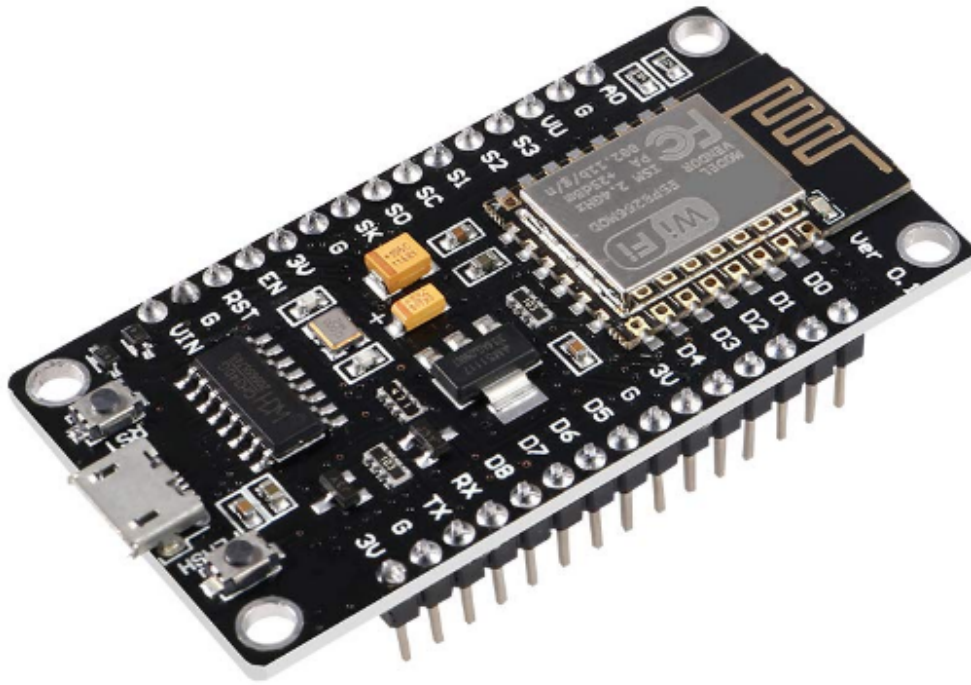


Figure 4: ESP8266 Microcontroller Module

OLED Display (1.3 inch)

The 1.3-inch OLED display is a compact and energy-efficient module used to visualize environmental sensor data in real-time.

It utilizes organic light-emitting diode technology to provide high contrast and crisp visibility, making it ideal for embedded applications where readability is important.

In our weather station, the OLED display is powered by the 3.3V supply from the ESP8266 and communicates with the microcontroller via the I2C protocol, sharing the communication lines with the BMP180 sensor.

The display is programmed to present real-time readings of temperature, humidity, atmospheric pressure, and altitude in a structured and user-friendly format.

Its low power consumption and reliable performance make it suitable for portable battery-powered systems.

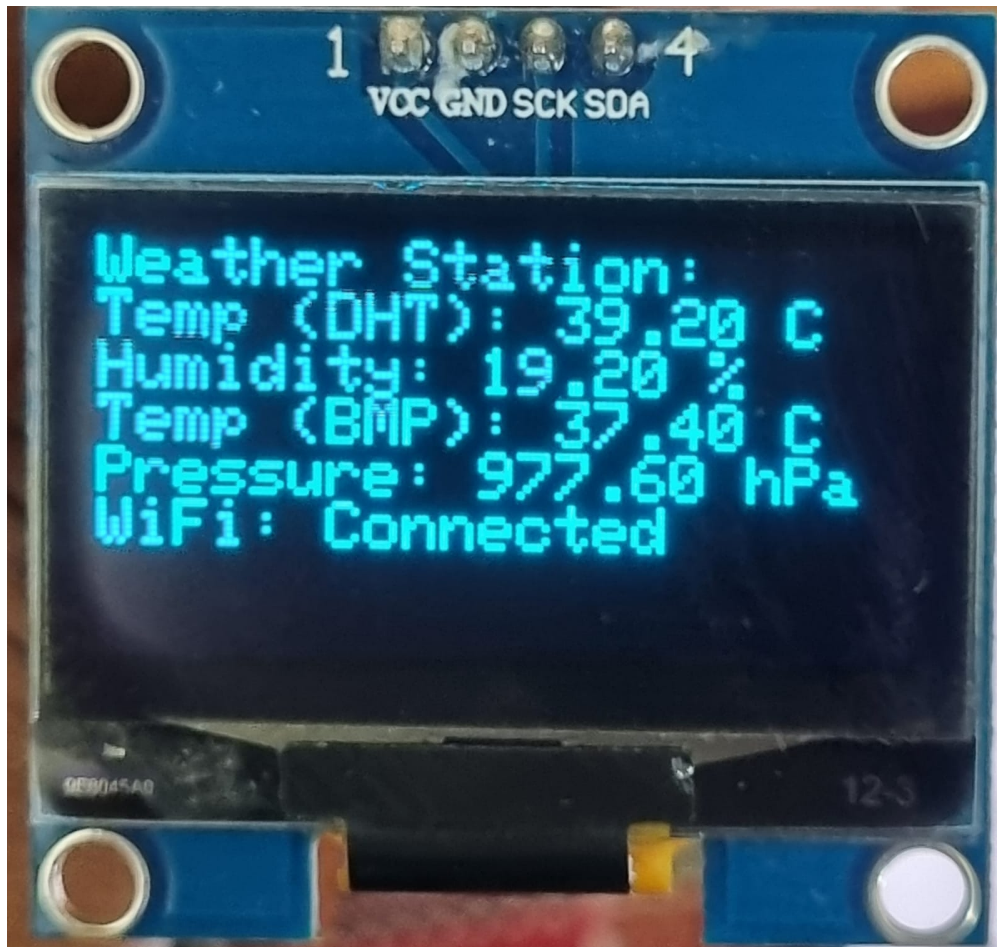


Figure 5: 1.3-inch OLED Display Module

TP4056 Lithium-ion Battery Charging Module

The TP4056 is a linear charger designed for single-cell lithium-ion batteries. It provides constant current and constant voltage charging with built-in safety features like overcharge protection and temperature monitoring.

In this project, the TP4056 charges the 18650 Li-ion battery from a USB power source, ensuring safe and efficient battery management.

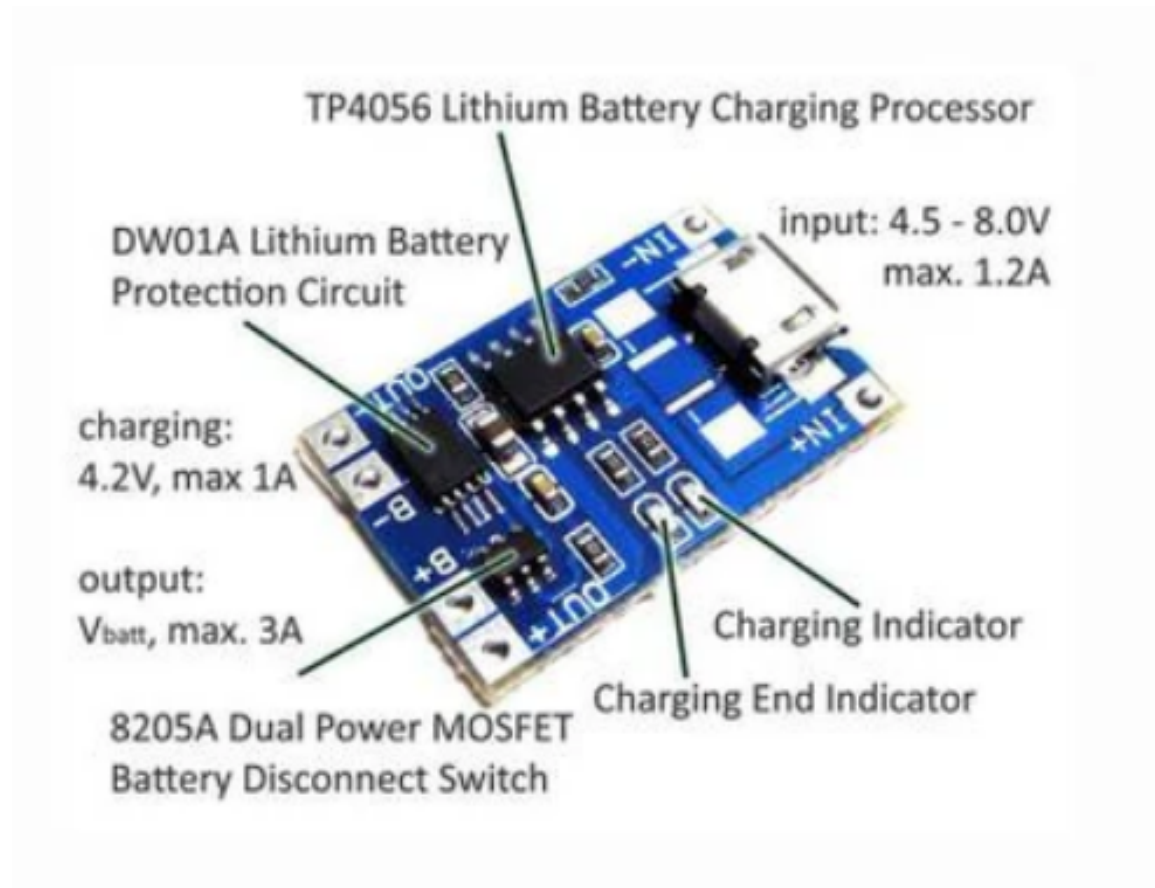


Figure 6: TP4056 Lithium-ion Battery Charging Module

MT3608 Boost Converter

The MT3608 is a high-efficiency step-up (boost) DC-DC converter module that increases lower input voltages (e.g., from a 3.7V Li-ion battery) to a stable 5V output.

It plays a critical role in our weather station by ensuring that the ESP8266 and other components receive a consistent power supply, even as the battery voltage drops during use.

The MT3608 operates using a simple two-wire input and output configuration and includes an onboard potentiometer to adjust the output voltage.

Its compact design and reliable voltage regulation make it ideal for battery-powered embedded systems, enhancing performance and extending runtime.

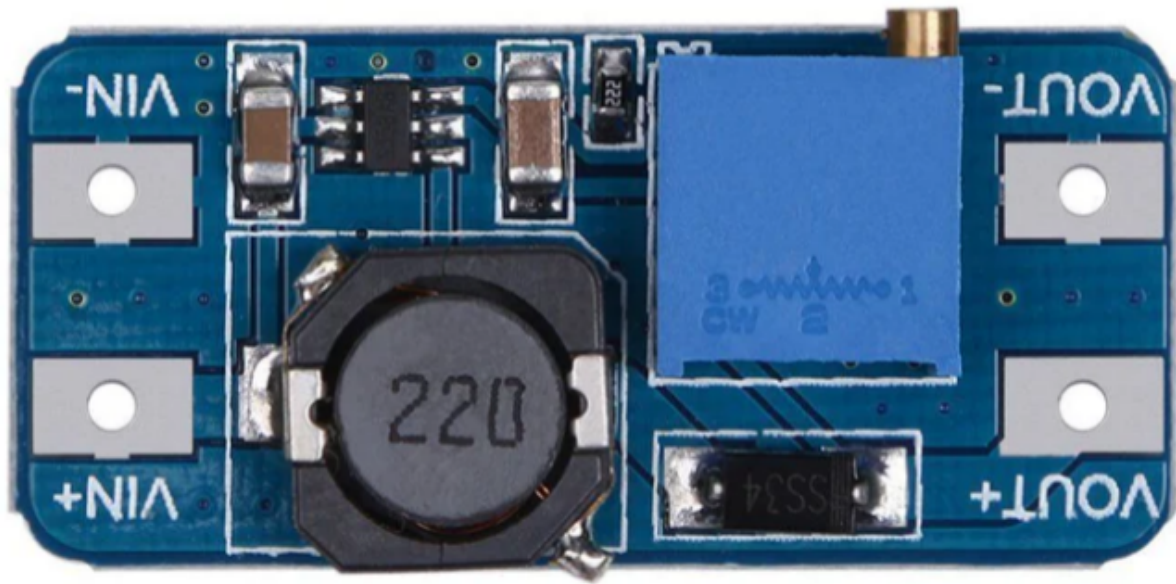


Figure 7: MT3608 DC-DC Boost Converter Module

Working

The Cloud-Based Weather Station functions as a compact, low-power embedded system capable of capturing, displaying, and transmitting environmental data. Below is a step-by-step overview of its working:

1. **Power Supply:** The system is powered by a 3.7V Li-ion battery, which is managed using a TP4056 charging module and boosted to 5V by the MT3608 boost converter to power the ESP8266 and peripheral components.
2. **Sensor Data Collection:**
 - The DHT11 sensor continuously measures ambient temperature and relative humidity.
 - The BMP180 sensor collects atmospheric pressure data and calculates altitude based on barometric pressure.
3. **Data Processing:** The ESP8266 microcontroller reads the sensor data through its GPIO and I2C interfaces. It processes this data to ensure validity and prepares it for display and transmission.
4. **Display Output:** The real-time sensor readings are displayed on the 1.3-inch OLED screen using the I2C interface. The user can view temperature, humidity, pressure, and altitude directly on the device.
5. **WiFi Connection:** The ESP8266 attempts to connect to a pre-configured WiFi network using the credentials hardcoded in the firmware.
6. **Cloud Integration:** Upon successful connection, the device formats the collected data into a JSON object and sends it to a cloud server via HTTPS using a POST request. This enables remote monitoring and logging of weather data.
7. **Looping Functionality:** The entire process runs in a continuous loop with periodic delays, ensuring that updated readings are consistently collected, displayed, and uploaded to the cloud.

This streamlined architecture allows the weather station to operate autonomously, providing real-time environmental monitoring and future scalability for IoT-based smart systems.

Schematic Diagram

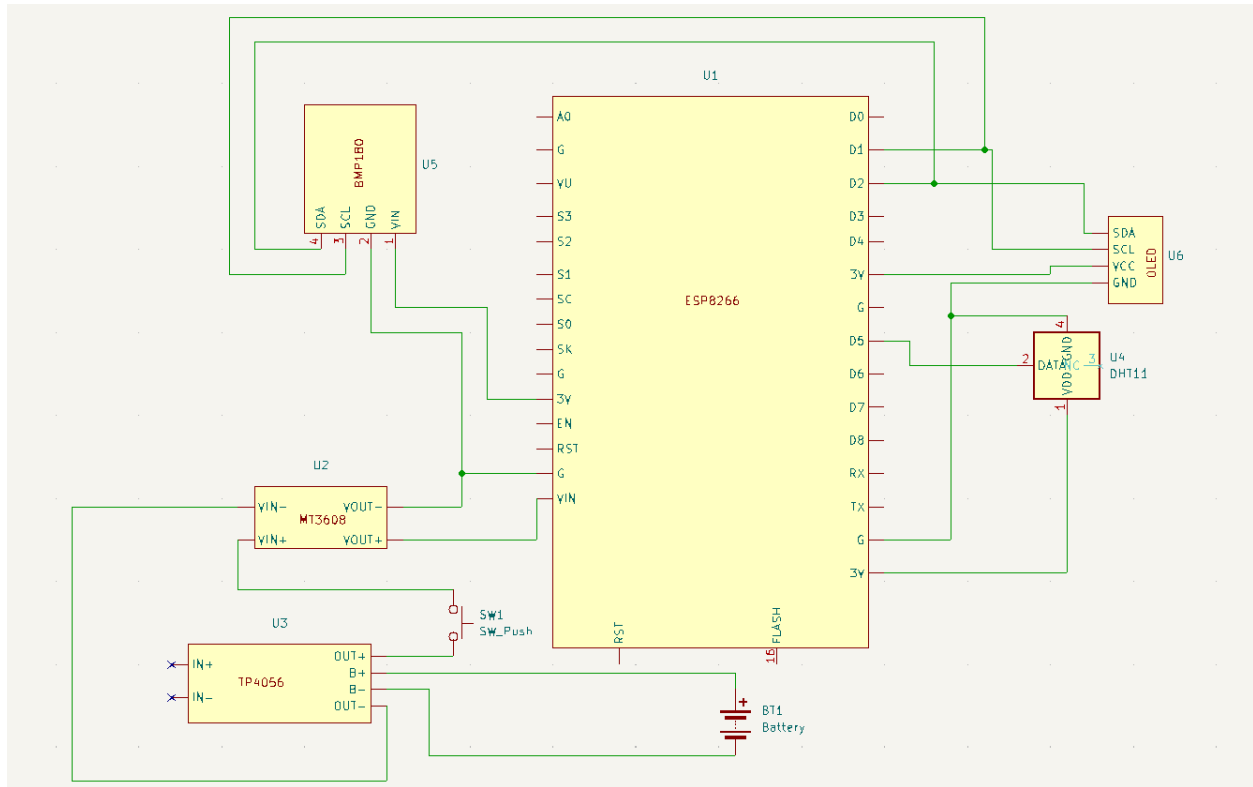


Figure 8: Weather Station

Arduino IDE Code

1. WiFi and Server Configuration

Code:

```
cpp
// === WiFi configuration ===
const char* ssid = "wpa";
const char* password = "1234567890";

// === Server configuration ===
const char* serverUrl = "https://weatherwatchpro-lkqrb2gq4-
  webrookie0s-projects.vercel.app/api/weather";
```

Explanation:

This section defines the WiFi SSID and password for connecting the ESP8266 to a wireless network. The server URL specifies the API endpoint where the weather data will be sent.

2. Sensor Initialization

Code:

```
cpp
#define DHTPIN D5
#define DHTTYPE DHT11
#define OLED_I2C_ADDRESS 0x3C

DHT dht(DHTPIN, DHTTYPE);
Adafruit_BMP085_Unified bmp = Adafruit_BMP085_Unified(10085);
Adafruit_SH1106G display = Adafruit_SH1106G(128, 64, &Wire, -1);
```

Explanation:

This section initializes the sensors (DHT11 for temperature and humidity, BMP180 for temperature and pressure) and the OLED display. The pins and sensor types are specified for proper configuration.

3. WiFi Connection Handling

Code:

```
cpp
WiFi.begin(ssid, password);

int wifiAttempts = 0;
while (WiFi.status() != WL_CONNECTED && wifiAttempts < 20) {
    delay(500);
    wifiAttempts++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("WiFi_connected");
} else {
    Serial.println("WiFi_connection_failed");
}
```

Explanation:

This part attempts to connect the ESP8266 to the WiFi network. It retries up to 20 times, with a delay between attempts. If successful, it confirms the connection; otherwise, it reports a failure.

4. Data Reading and Display

Code:

```
cpp
float humidity = dht.readHumidity();
float temp_dht = dht.readTemperature();

if (bmp_available) {
    sensors_event_t event;
    bmp.getEvent(&event);
    if (!isnan(event.pressure)) {
        pressure = event.pressure;
        bmp.getTemperature(&temp_bmp);
    }
}

Serial.println("Temperature:_" + String(temp_dht) + "_C");
Serial.println("Humidity:_" + String(humidity) + "_%");
```

Explanation:

This segment reads temperature and humidity from the DHT11 sensor and temperature and pressure from the BMP180 sensor. It displays the data on the serial monitor if the readings are valid.

5. Sending Data to Server

Code:

```
cpp
String jsonString;
serializeJson(jsonDoc, jsonString);

https.addHeader("Content-Type", "application/json");
int httpCode = https.POST(jsonString);
if (httpCode == HTTP_CODE_OK || httpCode == HTTP_CODE_CREATED) {
    String response = https.getString();
    Serial.println("Success!_Response:_" + response);
} else {
    Serial.println("HTTP_Error:_" + String(httpCode));
}
```

```
https.end();
```

Explanation:

This part of the code serializes the sensor data into a JSON format and sends it to the server using HTTPS. If the response indicates success, it prints the response; otherwise, it prints an error code.

6. Displaying Data on OLED

Code:

```
cpp
display.clearDisplay();
display.setCursor(0, 0);
display.println("Weather Station:");
display.print("Temp: ");
display.print(temp_dht);
display.println("C");
display.print("Humidity: ");
display.print(humidity);
display.println("%");
display.display();
```

Explanation:

This section updates the OLED display with the latest temperature and humidity readings. It clears the screen, sets the cursor position, and prints the sensor values in a readable format.

Website Walkthrough

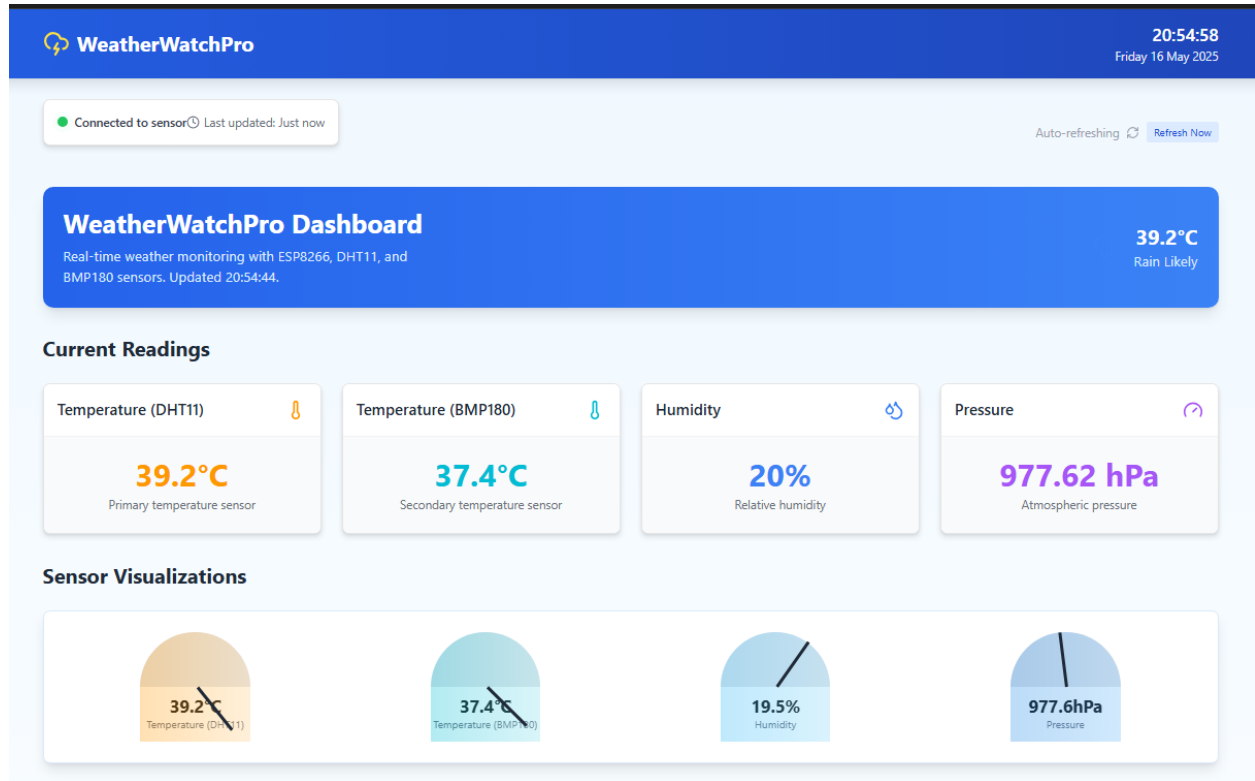


Figure 9: Dashboard showing real-time temperature, humidity, and pressure readings

The WeatherWatchPro website provides a clean and responsive user interface for real-time monitoring of environmental data collected via ESP8266. Figures 9 and 10 show the live dashboard interface. The key components of the interface are as follows:

1. Header

The top header includes:

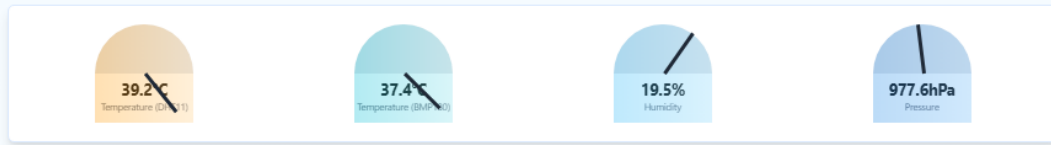
- **Website title and logo** on the left.
- **Real-time clock and date** on the right.

2. Sensor Status Panel

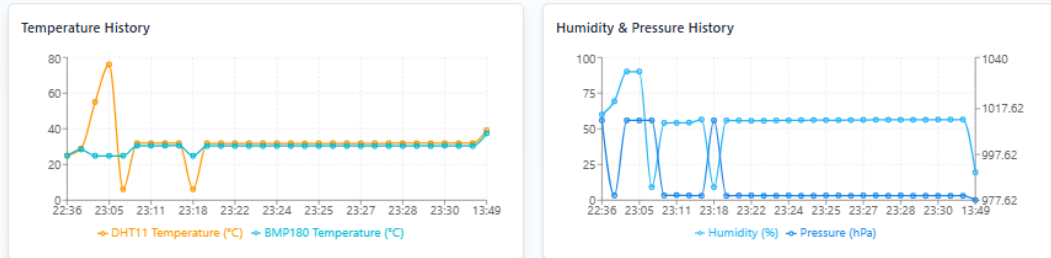
Below the header, a sensor status panel displays:

- Connection status with the sensor (e.g., *Connected to sensor*).
- Last updated timestamp for data freshness.

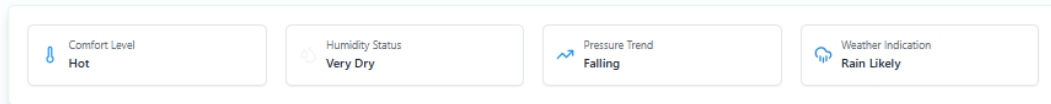
Sensor Visualizations



24-Hour History



Weather Interpretation



Device Information

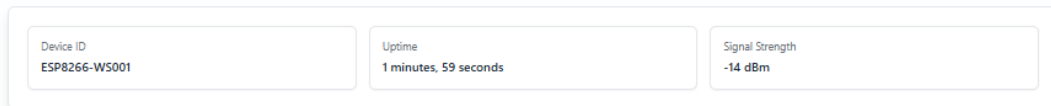


Figure 10: Detailed sensor visualizations using semi-circular gauges

3. Dashboard Banner

This prominent banner contains:

- Project title: *WeatherWatchPro Dashboard*.
- Brief description of sensor setup.
- Live weather status and summary temperature (e.g., *Rain Likely, 39.2°C*).

4. Current Readings

This section shows numerical readings from different sensors:

- **Temperature (DHT11):** Primary temperature in orange (e.g., 39.2°C).
- **Temperature (BMP180):** Secondary temperature in blue (e.g., 37.4°C).
- **Humidity:** Relative humidity in blue (e.g., 20%).
- **Pressure:** Atmospheric pressure in violet (e.g., 977.62 hPa).

5. Sensor Visualizations

The dashboard includes semi-circular gauge meters for each sensor to provide a visual representation of:

- DHT11 Temperature
- BMP180 Temperature
- Humidity
- Pressure

These allow quick at-a-glance understanding of current environmental conditions.

Website Development

Abstract

This project report presents the development of a weather station that collects environmental data using the ESP8266 module and sends it to a backend server. The frontend is built using React.js, HTML, and Tailwind CSS, while the backend utilizes Express.js. The collected data is displayed on a responsive web interface.

Introduction

With the increasing need for real-time weather monitoring, this project aims to develop a weather station that collects data such as temperature, humidity, and other parameters. The data is sent to a backend server, processed, and displayed on a user-friendly web application.

System Architecture

The weather station comprises three primary components:

- **Hardware:** ESP8266 microcontroller integrated with weather sensors.
- **Backend:** An Express.js server deployed at <https://weatherwatchpro-cqsvmw3ap-webrookie0s-p.vercel.app/api/weather>, which logs data received from POST requests.
- **Frontend:** A React.js application styled with Tailwind CSS that visualizes the logged data.

Implementation

- The ESP8266 module collects weather data and sends POST requests to the backend server.
- The backend processes the data and logs it in a database (if applicable).
- The frontend fetches the logged data from the server and displays it dynamically on the website.

Results

The system successfully collects, transmits, and displays weather data in real time. The deployment of the backend server ensures data availability and remote monitoring.

Conclusion

This weather station efficiently demonstrates the integration of IoT and web technologies. Future improvements may include adding more sensors and enhancing data visualization.

Conclusion and Learning

This project successfully demonstrated the design and implementation of a Cloud-Based Weather Station using the ESP8266 microcontroller and environmental sensors. The system is capable of real-time data acquisition, local display on an OLED screen, and remote data transmission to a cloud server for online monitoring.

Key outcomes include:

- Integration of multiple sensors (DHT11, BMP180) for comprehensive weather data collection.
- Effective use of the ESP8266's WiFi capabilities for seamless cloud connectivity.
- Development of a responsive web dashboard to visualize sensor data remotely.
- Practical experience in embedded system design, sensor interfacing, and IoT communication protocols.
- Power management using battery charging (TP4056) and boost conversion (MT3608) to ensure consistent system operation in portable conditions.

Through this project, we gained hands-on experience in:

- Programming microcontrollers and working with the Arduino IDE.
- Configuring and using various digital sensors over GPIO and I2C protocols.
- Building and testing circuits on Zero PCBs and integrating multiple hardware components.
- Designing and deploying full-stack web applications to handle real-time data.
- Collaborating as a team in planning, coding, debugging, and documentation.

This workshop has deepened our understanding of embedded systems and IoT-based development. The knowledge and skills acquired will be invaluable in future academic and professional projects involving smart systems and environmental sensing technologies.